MINISTRY OF EDUCATION AND TRAINING
NATIONAL ACADEMIC RESEARCH INSTITUTE
FACULTY OF INFORMATION TECHNOLOGY AND CYBERSECURITY

***

ACADEMIC CAPSTONE RESEARCH REPORT
CRYPTOGRAPHY AND INFORMATION ASSURANCE

REAL-WORLD APPLICATIONS OF RSA: CRYPTOGRAPHIC FOUNDATIONS, PRACTICAL IMPLEMENTATION, AND OFF-CHAIN BLOCKCHAIN AUTHENTICATION

RESEARCH PROJECT MEMBER LIST / STUDENT IDENTIFICATION

No.	Full Name	Student ID
1	.....	.....
2	.....	.....
3	.....	.....
4	.....	.....
5	.....	.....
6	.....	.....
7	.....	.....

HANOI, 2026

Real-World Applications of RSA: Cryptographic Foundations, Practical Implementation, and Off-Chain Blockchain Authentication

Academic Research Paper in Cryptography, Cybersecurity, and Blockchain Systems

Abstract— While public blockchains primarily adopt elliptic-curve cryptography (ECC) to minimize ledger footprint, RSA remains widely used in enterprise Public Key Infrastructure (PKI) and backend service authentication. This report investigates the practical trade-offs of RSA across theoretical foundations, system integration, and empirical performance. We contrast RSA-OAEP encryption with RSA-PSS digital signatures, analyze why distinct trust architectures favor RSA or ECC, and evaluate an educational prototype that uses RSA-PSS with SHA-256 to authenticate off-chain cryptocurrency withdrawal requests. Benchmarks across RSA (2048, 3072, 4096 bits) and ECDSA (secp256k1) show an asymmetric performance profile: although ECDSA signing is 5.82× faster than RSA-3072 (0.2553 ms vs. 1.4849 ms), RSA public exponent verification (e = 65,537) is 6.18× faster than ECDSA at the comparable 128-bit security level (0.0392 ms vs. 0.2423 ms). Adversarial tests show that combining digital signatures with timestamp validity windows and nonce tracking prevents unauthorized payload tampering and replay attempts. Testing against the bundled Project Wycheproof suite (108 vectors) confirms strict standard compliance, accepting all 63 valid test cases and rejecting all 45 invalid cases without error. Finally, we discuss key engineering limitations of the prototype, including in-memory state and the distinction between cryptographic validity and legal withdrawal authorization.

Keywords: *RSA Cryptography, RSA-PSS, Digital Signatures, PKI, Hybrid Encryption, Cryptocurrency Security, Off-chain Authentication, Empirical Evaluation.*

1. Introduction and Research Scope

Ron Rivest, Adi Shamir, and Leonard Adleman introduced the RSA cryptosystem in 1977, presenting a practical solution to the public-key distribution problem [1]. Standardized in PKCS #1 / RFC 8017 [2], RSA separates the private key transformation used for decryption and signing from the public key transformation used for encryption and verification. Over four decades later, RSA continues to secure web traffic, software updates, and enterprise authentication.

In modern blockchain environments, signature scheme selection involves conflicting engineering goals. Public blockchains such as Bitcoin and Ethereum rely on elliptic-curve cryptography (ECDSA and Schnorr over secp256k1) because small key and signature footprints reduce consensus storage costs. This widespread adoption has led to an assumption that RSA has become obsolete. However, algorithm selection depends on system boundaries. While ECC is well-suited for distributed ledger consensus, RSA offers faster public-key verification and mature Hardware Security Module (HSM) support, making it useful in high-throughput off-chain service pipelines.

2. RSA Fundamentals and Real-World Applications

2.1 Mathematical Foundations and Operating Mechanics

RSA relies on the integer factorization problem—the computational difficulty of recovering prime factors p and q from a large composite modulus n = p · q. Key generation begins by selecting two distinct large primes p and q. The system computes modulus n and Euler's totient function $\varphi(n) = (p - 1)(q - 1)$. The public exponent e is chosen such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$, with e = 65,537 ($2^{16} + 1$) selected as the industry standard. The private exponent d is the modular multiplicative inverse: $d \equiv e^{-1} \pmod{\varphi(n)}$, meaning $e \cdot d \equiv 1 \pmod{\varphi(n)}$. The public key is (n, e), while the private key is (n, d), often stored in Chinese Remainder Theorem (CRT) form (p, q, dP, dQ, qInv) to accelerate private exponentiation. Correctness follows from Euler's theorem: if gcd(m, n) = 1, then $m^{\varphi(n)} \equiv 1 \pmod{n}$. Since $e \cdot d = 1 + k \cdot \varphi(n)$, applying both exponents recovers the message: $(m^e)^d \equiv m^{(1 + k \cdot \varphi(n))} \equiv m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}$. However, textbook RSA—encrypting via $c = m^e \bmod n$ or signing via $s = m^d \bmod n$—is insecure. Because textbook RSA is deterministic, an attacker can identify identical messages. It is also multiplicatively homomorphic: $(m_1 \cdot m_2)^d \equiv m_1^d \cdot m_2^d \pmod{n}$. This algebraic relationship allows an attacker to combine valid signatures on m_1 and m_2 into a forged signature for $m_1 \cdot m_2$ without knowing private exponent d.

2.2 Distinction Between RSA Encryption and RSA Digital Signatures

RSA supports two distinct operational primitives designed for opposing security objectives: (1) Asymmetric Encryption (RSA-OAEP): Optimal Asymmetric Encryption Padding (RFC 8017 Section 7.1 [2]) formats plaintext before modular exponentiation $c = (EM)^e \bmod n$ to provide confidentiality. OAEP applies a two-round Feistel structure parameterized by a cryptographic hash function (such as SHA-256) and a Mask Generation Function (MGF1) seeded with random bytes r. This randomization ensures indistinguishability under adaptive chosen-ciphertext attack (IND-CCA2) in the Random Oracle Model (ROM) [14, 15], preventing padding oracle attacks like Bleichenbacher's attack on PKCS #1 v1.5. (2) Digital Signatures (RSA-PSS): The Probabilistic Signature Scheme (RFC 8017 Section 8.1 [2]) formats message digests before private exponentiation $s = (OS2IP(EM))^d \bmod n$ to guarantee authenticity and non-repudiation. EMSA-PSS incorporates the message digest H = Hash(m), a random salt of length sLen (standardized to 32 bytes for SHA-256), and MGF1 masking. RSA-PSS achieves tight existential unforgeability under adaptive chosen-message attack (EUF-CMA) in the ROM [16]. Because the padding schemes, security goals, and mathematical proofs differ completely, FIPS 186-5 [4] strictly mandates cryptographic key separation: an RSA key pair assigned to digital signatures must never be used for encryption.

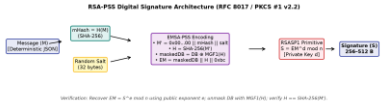


Figure 1. Operational workflow of the RSA-PSS probabilistic digital signature scheme (RFC 8017 / FIPS 186-5).

2.3 Real-World Application Breadth

In production environments, RSA is primarily deployed across three domains: (1) Hybrid Encryption and Key Transport: In protocols such as TLS 1.2, RSA-OAEP encrypts ephemeral symmetric session keys (e.g., AES-256-GCM), combining asymmetric key exchange with fast symmetric data transfer. (2) TLS 1.3 Endpoint Authentication: RFC 9846 [3] deprecated static RSA key exchange in favor of ephemeral Diffie-Hellman ((EC)DHE) to enforce Forward Secrecy. However, RSA remains common for server CertificateVerify authentication, where modern implementations use RSA-PSS for certificate verification. (3) Enterprise PKI and Code Signing: Operating system bootloaders and enterprise Certificate Authorities (CAs) rely heavily on RSA-3072 and RSA-4096 root certificates due to broad legacy hardware compatibility.

3. Why RSA? Security Purpose and Comparative Evaluation

Table 1 systematically compares RSA configurations with modern elliptic curve alternatives: ECDSA (secp256k1) [11], Schnorr (BIP 340) [7], and Ed25519 [10]. Security strengths are classified according to NIST SP 800-56B Rev. 2 [5] and NIST SP 800-57 Part 1 Rev. 5 [6].

Evaluation Metric	RSA-2048	RSA-3072	ECDSA (secp256k1)	Schnorr (BIP 340)	Ed25519 (RFC 8032)
Hard Problem	Factoring	Factoring	EC Discrete Log	EC Discrete Log	Edwards EC-DLOG
Security Strength	112 bits [5]	128 bits [5]	~128 bits [6]	128 bits [6]	128 bits [6]
Key Size / Sig Size	Modulus: 256 B Sig: 256 B	Modulus: 384 B Sig: 384 B	PubKey: 33 B (comp) Sig: 70–72 B (DER)	PubKey: 32 B (x-only) Sig: 64 B (BIP341)	PubKey: 32 B Sig: 64 B [10]
Relative Verify Speed	Empirical: 0.021 ms (11.32× vs ECDSA)	Empirical: 0.039 ms (6.18× vs ECDSA)	Empirical: 0.242 ms (Baseline: 1.0×)	Batch verify supported; not benchmarked [7]	High efficiency in lit. [10]; not benchmarked

Table 1. Systematic comparison of asymmetric cryptographic algorithms across security and operational metrics.

3.1 Storage and Verification Trade-Offs in Blockchain Consensus Layers

Public blockchains must minimize ledger growth and bandwidth demand because every full node validates and stores all transaction signatures. In Bitcoin, a standard SegWit P2WPKH input uses a witness stack of roughly 107–109 bytes, consisting of a 70–72 byte DER signature and a 33-byte compressed public key [8, 11]. Using RSA-2048 for transaction authorization would require storing both the 256-byte modulus and the 256-byte signature. With length prefixes, this produces 519 bytes per input—adding over 410 witness bytes (~103 virtual bytes under BIP 141 rules [8]). For a typical 141-vB transaction, virtual size would grow by ~73% to 244 vB, increasing fee overhead. Bitcoin script also provides no native RSA verification opcodes. Ethereum exhibits similar constraints: consensus rules require ECDSA over secp256k1 for Externally Owned Account (EOA) transactions [12]. Although the EVM includes a modular exponentiation precompile at address 0x05 (EIP-198), it only calculates B^E mod M. Implementing full RSA-PSS verification—including EMSA-PSS unmasking, MGF1 hashing, and salt checks—in EVM bytecode consumes substantial gas, making on-chain RSA validation impractical. In contrast, BIP 340 Schnorr signatures remain 64 bytes and support multi-signature aggregation via MuSig2 (BIP 327 [9]), collapsing n signers into a single signature.

3.2 Operational Rationale for RSA in Off-Chain Backends

While poorly suited for on-chain consensus, RSA offers practical benefits in off-chain exchange backends: (1) Fast Public Exponent Verification: Using public exponent $e = 65{,}537 = 2^{16} + 1$ (Hamming weight 2), modular exponentiation requires only 16 squarings and 1 multiplication (17 operations total). This asymmetry allows edge gateways to verify signatures with minimal computational latency under heavy traffic. (2) Hardware Security Module Maturity: Banking and enterprise HSMs provide long-established, certified hardware acceleration for RSA operations. (3) Absence of Scalar Nonce Fragility: Standard ECDSA implementations face severe risks if the signing nonce k is reused or leaks even fractional bit bias, which algebraically reveals private key d_A. RSA-PSS does not employ a secret scalar nonce, avoiding this class of implementation failure. However, software must use base blinding ($x' = x \cdot r^e \bmod n$) to defend against timing side channels. Furthermore, blinding does not stop Chinese Remainder Theorem (CRT) hardware fault injection (the Bellcore attack [7]), where a single bit error during signing reveals private prime factors via $\gcd(s - s', n) \in \{p, q\}$. Defending against CRT fault injection requires verifying the signature ($s^e \equiv m \bmod n$) before releasing it, or employing redundant hardware checking.

4. Proposed System Design and Implementation

To study RSA in an operational setting, we implement an educational off-chain architecture: authenticating cryptocurrency withdrawal requests using RSA-PSS and SHA-256 before forwarding them to hot-wallet dispatchers. Operational Scope: This system functions entirely within an internal service boundary. It verifies administrative approval for withdrawals prior to blockchain broadcast. It does not replace on-chain ECDSA or Schnorr signatures on Bitcoin or Ethereum, nor does it replicate proprietary exchange infrastructure.

4.1 Architecture and Component Decomposition

As shown in Figure 2, the pipeline separates request handling into four tiers: 1. Withdrawal Service: Collects withdrawal parameters, including target network, recipient address, atomic amount (wei), a UUIDv4 nonce, and a UNIX timestamp. 2. Signing Component: Represents an isolated service or HSM that serializes the payload deterministically, computes its SHA-256 digest, and generates an RSA-PSS signature. 3. Verification Gateway: Receives the signed request, looks up the approved RSA public key, and verifies the PSS signature. 4. Validation Engine: Checks timestamp freshness (allowing a ±300-second clock skew window) and validates nonce uniqueness against an anti-replay store before permitting hot-wallet execution.

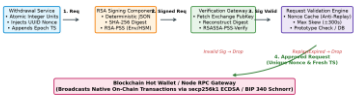


Figure 2. Proposed off-chain cryptocurrency withdrawal authentication architecture using RSA-PSS.

4.2 Deterministic JSON Serialization and Pipeline Algorithm

Cryptographic verification fails if microservices serialize JSON keys or whitespace differently. The prototype enforces canonical byte representations by sorting dictionary keys lexicographically, removing extraneous whitespace, and encoding using UTF-8. While this provides consistent byte sequences for our specific schema, it is not a full implementation of RFC 8785 JSON Canonicalization Scheme (JCS) floating-point rules. Following RFC 8017 Section 9.1 [2], EMSA-PSS encoding specifies emBits = modBits − 1 (2047 bits for RSA-2048, emLen = 256 bytes; salt length sLen = 32 bytes). Algorithm 1 summarizes the complete verification workflow.

```
Algorithm 1: Off-Chain Withdrawal Authentication Pipeline
Input:  Request R = {id, net, addr, amt_atomic, nonce, ts}, Private Key SK_rsa, Public Key PK_rsa
Output: Decision ∈ {ACCEPT, REJECT_SIG, REJECT_REPLAY, REJECT_EXPIRED}
1:  M_canon ← DeterministicJSON(R)                    // Lexicographical keys, compact separators, UTF-8
2:  EM ← EMSA-PSS-ENCODE(M_canon, emBits=modBits−1, sLen=32)  // RFC 8017 PSS encoding
3:  σ ← RSASP1(SK_rsa, OS2IP(EM))                     // Signature generation
4:  // Gateway Verification Phase
5:  valid_sig ← RSASSA-PSS-Verify(PK_rsa, M_canon, σ, SHA-256, sLen=32)
6:  if ¬ valid_sig then return REJECT_SIG              // Tampering / invalid key detected
7:  if |CurrentTime() − R.ts| > Δ_skew then return REJECT_EXPIRED
8:  if R.nonce ∈ NonceStore then return REJECT_REPLAY  // Replay detected
9:  NonceStore.Add(R.nonce); return ACCEPT
```

4.3 Prototype Scope and Implementation Limitations

A rigorous assessment must acknowledge the gap between this experimental prototype and production-grade exchange security: (1) In-Memory Nonce Tracking: The prototype stores seen nonces in an in-memory Python set. A production system must use distributed atomic storage (such as Redis SETNX or SQL unique constraints) with automated TTL eviction to handle service restarts and prevent race conditions across multiple nodes. (2) Address Formatting: The prototype performs basic sanity checks on address format. Production systems require strict checksum validation (e.g., EIP-55 for Ethereum or Bech32 for Bitcoin). (3) Key Storage and Lifecycle: In our tests, private keys reside in application memory. Production deployments require FIPS 140-3 HSM enclaves, automated key rotation schedules, and Certificate Revocation Lists (CRL) or OCSP responder infrastructure. (4) Signature Validity vs. Authorization: A valid RSA signature proves that the request was signed by the corresponding private key and that the payload was not modified in transit. It does not prove that the requester possesses the legal authority or KYC clearance to withdraw funds, which requires separate business logic and identity verification tiers.

5. Experimental Results and Analysis

5.1 Adversarial Security Testing

We tested the verification pipeline against five adversarial test scenarios using rsa_pss_experiment.py: (1) Valid Request: A legitimate 12.5 ETH withdrawal request verified successfully (ACCEPTED); (2) Tampered Amount: Modifying the payload amount to 50.0 ETH invalidated the signature (REJECTED); (3) Tampered Recipient: Altering the destination address caused signature verification to fail (REJECTED); (4) Unauthorized Public Key: Verifying the signature against a different public key failed (REJECTED); (5) Replay Attempt: Resubmitting an identical, previously accepted payload was blocked by the nonce cache (REJECTED). Supplementary mutation tests (test_m05_adversarial.py) evaluated signature corruption, single-bit flips, truncated buffers, and empty inputs. In all cases, the verification engine rejected invalid data safely without unhandled exceptions.

5.2 Bundled Wycheproof Test Suite Validation

To evaluate standards compliance against third-party test data, we ran our verification engine against the bundled Project Wycheproof RSA-PSS 2048-bit test suite (SHA-256, MGF1-SHA256, 32-byte salt; snapshot v0.9, rsa_pss_2048_sha256_mgf1_32_test.json) [13]. The suite contains 108 test vectors: 63 valid test cases and 45 invalid test cases. The engine achieved 100% agreement: all 63 valid signatures were accepted, all 45 invalid signatures were rejected, with 0 skipped and 0 harness errors.

5.3 Empirical Benchmarking and Performance Analysis

We benchmarked the prototype using Python's cryptography library (v50.0.2) backed by OpenSSL 4.0.3 on Windows 11 (Intel Core i5-12400F, 2.50 GHz). Measurements were collected over 10 key generation trials and 100 signing and verification trials across RSA-2048, RSA-3072, RSA-4096 (e = 65,537, SHA-256, salt = 32B), and ECDSA (secp256k1, SHA-256). Table 2 and Figure 3 report the median operational latencies.

Algorithm	Security Level	Keygen Median	Sign Median	Verify Median	Sig Size	Internal Asymmetry
RSA-2048	112 bits [5]	28.98 ms	0.4736 ms	0.0214 ms	256 B	22.13× (Sign / Verify)
RSA-3072	128 bits [5]	172.3547 ms	1.4849 ms	0.0392 ms	384 B	37.88× (Sign / Verify)
RSA-4096	152 bits [5]	359.638 ms	3.2927 ms	0.0581 ms	512 B	56.67× (Sign / Verify)
ECDSA-secp256k1	~128 bits [6]	0.2672 ms	0.2553 ms	0.2423 ms	71 B (DER sample)	1.05× (Sign / Verify)

Table 2. Empirical benchmark measurements (Intel 12th Gen / Python 3.13 / OpenSSL 4.0.3, median of 10 keygen / 100 crypto trials, zero warm-up).

Performance Discussion: (1) RSA Operational Asymmetry: For all tested RSA key sizes, verification is substantially faster than signing: 22.13× for RSA-2048, 37.88× for RSA-3072, and 56.67× for RSA-4096. This asymmetry occurs because the public exponent e = 65,537 has a low Hamming weight, requiring only 17 modular operations, whereas private exponent d requires modular exponentiation over the full modulus bit length. (2) Comparative Profile at 128-Bit Security: Comparing RSA-3072 and ECDSA-secp256k1 (both offering approximately 128 bits of classical security): ECDSA signing is 5.82× faster than RSA-3072 (0.2553 ms vs. 1.4849 ms), and ECDSA key generation is over 645× faster (0.2672 ms vs. 172.3547 ms). In contrast, RSA-3072 verification (0.0392 ms) is 6.18× faster than ECDSA (0.2423 ms). At 0.0392 ms per verification, a single CPU core theoretically processes ~25,510 RSA-3072 signatures per second (compared to ~4,127 for ECDSA). These raw cryptographic measurements isolate algorithm latency; production throughput is further bounded by network I/O, serialization, and database lookups. Relative ratios also reflect microarchitectural traits compared to an earlier AMD benchmark run (8.22× verify / 3.87× sign).

6. Conclusion

This study examined the practical trade-offs of the RSA cryptosystem in modern computing environments. Although elliptic-curve cryptography is the clear choice for blockchain consensus layers where compact signature size directly reduces ledger fees, RSA retains a strong role in off-chain systems. Thanks to its small public exponent, RSA-PSS delivers verification speeds several times faster than ECDSA, making it attractive for high-volume gateway authentication and enterprise PKI. Our prototype shows that combining RSA-PSS signatures with deterministic JSON serialization, timestamp checks, and nonce tracking creates an effective off-chain request filter. However, our evaluation also highlights that cryptographic signatures only solve part of the problem. A production-ready exchange requires stateful anti-replay databases, HSM key protection, and thorough identity governance. Looking ahead, because Shor's algorithm threatens both RSA and classical elliptic curves, long-term deployments must maintain crypto-agility and plan migration paths toward standardized post-quantum algorithms such as ML-DSA (FIPS 204) and SLH-DSA (FIPS 205).

7. References

[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," CACM, vol. 21, no. 2, pp. 120–126, 1978.
[2] K. Moriarty et al., "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, IETF, 2016. https://www.rfc-editor.org/rfc/rfc8017
[3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, IETF, 2026. https://www.rfc-editor.org/rfc/rfc9846
[4] NIST, "Digital Signature Standard (DSS)," FIPS 186-5, 2023. https://doi.org/10.6028/NIST.FIPS.186-5
[5] E. Barker et al., "Recommendation for Pair-Wise Key-Establishment Schemes," NIST SP 800-56B Rev. 2, 2019. https://doi.org/10.6028/NIST.SP.800-56Br2
[6] E. Barker, "Recommendation for Key Management: Part 1 – General," NIST SP 800-57 Pt. 1 Rev. 5, 2020. https://doi.org/10.6028/NIST.SP.800-57pt1r5
[7] P. Wuille, J. Nick, and T. Ruffing, "Schnorr Signatures for secp256k1," BIP 340, 2020. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki
[8] E. Lombrozo, J. Lau, and P. Wuille, "Segregated Witness (Consensus layer)," BIP 141, 2015. https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki
[9] J. Nick, T. Ruffing, and E. Jin, "MuSig2 for BIP340 Multi-Signatures," BIP 327, 2023. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki
[10] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017. https://www.rfc-editor.org/rfc/rfc8032
[11] Bitcoin Core Developers, "Bitcoin Developer Guide: Transactions," Bitcoin Project, 2026. https://developer.bitcoin.org/devguide/transactions.html
[12] Ethereum Foundation, "Ethereum Developer Documentation: Transactions and Signatures," 2026. https://ethereum.org/en/developers/docs/transactions/
[13] Google Security Research, "Project Wycheproof: Cryptographic Test Vectors (v0.9)," 2026. https://github.com/google/wycheproof
[14] E. Fujisaki et al., "RSA-OAEP is Secure under the RSA Assumption," Journal of Cryptology, vol. 17, no. 2, pp. 81–104, 2004.
[15] M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption," in EUROCRYPT '94, LNCS 950, pp. 92–111, 1995.
[16] M. Bellare and P. Rogaway, "The Exact Security of Digital Signatures," in EUROCRYPT '96, LNCS 1070, pp. 399–416, 1996.